

1. Создание ветки

main (или staging) - стабильная ветка

dev - общая ветка для разработки

dev1, dev2, dev3 - отдельные ветки разработчиков

feature/<feature-name> - ветки для отдельных фичей разработки

1.1. Создание общей ветки для разработки (dev) или разработчика (dev1)

```
git checkout -b dev1
```

```
git push -u origin dev1
```

1.2. Создание фичевой ветки

```
git checkout -b feature/<feature-name>
```

```
git add . | git add <Filename> | git add README.md
```

```
git commit -m "First commit"
```

```
git push -u origin feature/<feature-name>
```

2. Проведение коммитов

Если вы создадите новый файл или сделаете изменения, то они будут в рабочем пространстве и поэтому при переключении на другую ветку эти изменения будут присутствовать и там. Если закоммитить изменения, то они "останутся" в ветке.

```
git add . | git add <Filename> | git add README.md
```

```
git commit -m "<Commit description>"
```

3. Публикация ветки

Все закоммиченные изменения отправляем на сервер.

```
git push -u origin feature/<feature-name>
```

Тут возможны проблемы. -- rebase, -- no-rebase, merge???

4. Подготовка к Pull Request (feature/<feature-name> -> dev1)

Синхронизируемся с веткой dev1.

```
git pull origin/dev1
```

Тут возможны проблемы.

5. Создание Pull Request (feature/<feature-name> -> dev1)

Создаем PR, тут сравнение веток и вливанием коммитов из ветки

feature/<feature-name> в ветку dev1.

Аналогично из ветки dev1 в ветку dev, из ветки dev в ветку main или staging.

6. Проведение Pull Request(feature/<feature-name> -> dev1)

Ревьюеры проверяют, проводят PR. В результате изменения попадают в dev1.

—

Проблематика

You have diverged branches. (Расходящиеся ветки)

Что означает: Гит запутался как именно объединить ветки. Ситуация **diverged branches** (расходящиеся ветки): в вашей локальной ветке `branch1` есть коммиты, которых нет в `origin/branch2` и в `origin/branch2` есть новые коммиты, которых нет у вас.

Как воссоздать:

1. Находимся в ветке `branch1`
(есть отдельные коммиты в `branch1` и отдельные коммиты в `branch2`)
2. `git pull origin branch2`

Что делать: надо выбрать стратегию **Merge** (создать коммит слияния) или **Rebase** (перенести ваши коммиты поверх новых).

Выбор Merge - самый простой и безопасный способ.

Git создаст один специальный *"Merge commit"*, который свяжет две ветки.

```
git pull origin branch2 --no-rebase
```

Если появятся конфликты, Git предложит их исправить. Если нет - откроется текстовый редактор для ввода сообщения коммита (просто сохраните и закройте).

Выбор Rebase - чистая история.

Git возьмет ваши локальные коммиты из `branch1`, временно отложит их, накатит изменения из `branch2`, а потом приклеит ваши коммиты сверху.

```
git pull origin branch2 --rebase
```

Это делает историю линейной, что очень любят в командах. Но если ваши коммиты уже были в `origin/branch1`, то после этого придется делать `git push --force`.

Можно настроить, чтобы всегда был merge (рекомендуется)

```
git config --global pull.rebase false
```

-
-
-
-

—START—

1. Создать репозиторий на GitHub

Не добавляйте **README.md**, **.gitignore** и лицензии, чтобы избежать конфликтов.

2. Настроить Git в IntelliJ IDEA

Откройте **File** → **Settings** → **Version Control** → **Git**

Убедитесь, что путь к **git.exe** указан и нажмите **Test**.

3. Инициализировать Git в проекте

Если проект еще не под контролем Git:

1. Откройте **IntelliJ IDEA**.
2. Выберите **VCS** → **Enable Version Control Integration**.
3. В появившемся окне выберите **Git** и нажмите **OK**.

4. Подключить проект к удаленному репозиторию

```
git init
```

```
git remote add origin
```

https://github.com/ВАШ_ЛОГИН/ВАШ_РЕПОЗИТОРИЙ.git

Проверьте

```
git remote -v
```

5. Создать **.gitignore**

***.iml .idea/ out/ target/ *.log *.class**

6. Сделать первый коммит

```
git add .
```

```
git commit -m "Initial commit"
```

7. Отправить проект в GitHub

```
git branch -M main # Переименовываем в main (если нужно)
```

```
git push -u origin main
```

3. Определить, есть ли у веток общая история

```
git log --oneline --graph --all
```

Если ветки выглядят как две отдельные линии, значит, они не связаны.

-

6. Настройки

```
git config --global user.email  
git config user.email  
git log -1 (email of last commit)  
git config --global user.email "your@email.com"  
  
git commit --amend --author="Your Name <correct@email.com>"  
git push --force  
  
git config --get core.editor  
git config --global core.editor "nano"
```

7. Редакторы

VIM
(- Insert -) ESC - выйти из режима редактирования
:wq - выйти с сохранением
:q! - выйти без сохранения

NANO
touch filename
echo ".idea/" > .gitignore (create and fill)
echo ".git/" >> .gitignore (append text to file)
nano filename (CTRL + O - сохранить, CTRL + X - выйти)

8. Информация

```
git status  
git log  
git log --oneline  
git log --oneline --graph --all  
git shortlog -s -n (кто сколько строк добавил)  
git log --stat (статистика по коммитам)  
git log file (статистика по файлу)  
git blame file (построчная информация по файлу)  
git diff (сравнение с последний коммитом)  
git diff --cached (сравнение staged-изменений с последним коммитом)  
git branch (список всех веток)  
git reflog (журнал всех действий)  
-  
-  
-  
-  
-
```

1. Состояние "Detached HEAD" (Оторванная голова)

Это происходит, когда вы переключаетесь не на ветку, а на конкретный коммит (`git checkout <hash>`).

- **Проблема:** Вы находитесь вне какой-либо ветки. Если вы сделаете коммиты в этом состоянии, а потом переключитесь на другую ветку, ваши изменения будет очень сложно найти (они не принадлежат ни одной ветке и со временем могут быть удалены "мусорщиком" Git).
 - **Как лечить:** Если нужно сохранить изменения, создайте новую ветку прямо из этого состояния: `git checkout -b new-branch-name`.
-

2. Конфликты слияния (Merge Conflicts)

Самый частый «кошмар» новичка. Возникает, когда два человека изменили одни и те же строки в одном и том же файле.

- **Проблема:** Git не знает, чья правда, и останавливает процесс слияния, помечая файлы как `both modified`.
 - **Как лечить:** Открыть файлы, найти маркеры `<<<<<<, =====, >>>>>>`, вручную выбрать нужный код, сделать `git add` и `git commit`.
-

3. Ошибка "index.lock"

Иногда при попытке выполнить команду Git выдает ошибку: `Another git process seems to be running in this repository....`

- **Проблема:** Git создает файл блокировки, чтобы две команды не меняли репозиторий одновременно. Если предыдущая команда упала или зависла, файл остается.
 - **Как лечить:** Если вы уверены, что никакой Git-процесс сейчас не запущен, просто удалите файл вручную: `rm .git/index.lock`.
-

4. Огромные бинарные файлы в истории

Кто-то случайно закоммитил SQL-дамп на 2 ГБ или папку `node_modules`.

- **Проблема:** Даже если вы удалите файл следующим коммитом, он останется в истории папки `.git`. Репозиторий становится "тяжелым", `git clone` занимает вечность.
 - **Как лечить:** Использовать специальные инструменты вроде **BFG Repo-Cleaner** или команду `git filter-repo` для полной очистки истории от конкретного файла.
-

5. Проблемы с регистром в названиях файлов (Case Sensitivity)

Windows и macOS обычно не различают регистр (`File.kt` и `file.kt` для них одно и то же), а Linux — различает.

- **Проблема:** Вы переименовали `UserService.kt` в `Userservice.kt`. Windows может даже не заметить изменения, а на CI/CD сервере (Linux) проект упадет с ошибкой "File not found".
 - **Как лечить:** Настроить `git config core.ignorecase false` или переименовывать файлы через команду `git mv`.
-

6. "Засорение" рабочего каталога (Dirty Working Tree)

Вы хотите переключить ветку (`git checkout`), но Git не дает, так как у вас есть незакоммиченные изменения, которые конфликтуют с целевой веткой.

- **Проблема:** Нужно либо коммитить «мусор», либо всё удалять.
 - **Как лечить:** Используйте `git stash`. Это "карман", куда можно временно спрятать изменения, переключить ветки, а потом достать их обратно (`git stash pop`).
-

-
-
-
-
-
-
-
-
-

Команды с ветками (бранчами)

КОМАНДА	ОПИСАНИЕ
git branch "feature/task-35"	Создание новой локальной ветки
git branch -a	Посмотреть все ветки(локальные и origin)
git branch -vv	Показывает привязки локальных и origin веток
git branch -u origin/"dev"	Привязывает локальную и соответствующую origin
git branch -m "staging"	Переименовать ветку
git checkout "test"	Перейти до указанной ветки
git checkout -b "feature/task-35"	Создание новой локальной ветки и переход на нее
git checkout -d "feature/task-35"	Безопасное удаление ветки, если нет изменений
git checkout -D "feature/task-35"	(FORCE) Удаление ветки в независимости изменений

Команды с коммитами

КОМАНДА	ОПИСАНИЕ
git add <file-name>	
git add .	
git commit -m 'name of commit'	
git commit --amend	
git push	
git push origin <branch>	
git push -u origin <branch>	
git pull	
git pull origin <branch>	
git pull - -rebase [origin dev]	
git pull - -merge [origin dev] --no-rebase??	
git pull - -force [origin dev]	

Что такое Pull Request

Pull Request (PR) — это запрос на вливание изменений из одной ветки в другую.

Обычно:

- разработчик работает в своей feature-ветке
- пушит изменения в GitHub
- создает Pull Request
- код проходит review
- после проверки изменения вливаются в `main` / `develop`

При создании PR в GitHub:

- Compare & Pull Request
- описание изменений
- reviewers
- linked issues

Code Review (проверяют):

- архитектуру
- читаемость кода
- naming
- безопасность
- производительность
- баги
- соответствие style guide

Хороший PR:

- маленький
- делает одну задачу
- имеет понятное описание
- содержит понятные коммиты

Плохой PR:

- 5000 changed lines
- смешано 10 задач
- непонятное название
- случайные изменения

Временное откладывание изменений (stash)

`git stash` — временно убирает незакоммиченные изменения, чтобы переключиться на другую задачу.

Например:

- работаешь над feature
 - срочно нужно переключиться на hotfix
 - изменения еще не готовы к commit
-

Сохранить изменения

`git stash`

Git:

- сохранит изменения
 - очистит рабочую директорию
-

Посмотреть stash

`git stash list`

Вернуть изменения

`git stash apply` # stash остается

или:

`git stash pop` # stash удаляется после применения

Удалить stash

`git stash drop`
`git stash clear`

Сохранить stash с названием

```
git stash push -m "JWT auth draft"
```

Практический сценарий stash

```
git checkout feature/auth
```

```
# начали работу
```

```
git add .
```

```
# НЕ коммитим
```

```
# срочный баг
```

```
git stash
```

```
git checkout main
```

```
git checkout -b hotfix/login-bug
```

```
# исправили баг
```

```
git checkout feature/auth
```

```
git stash pop
```

Не хранить stash неделями

stash — временное решение.

Плохая практика:

- десятки stash
- непонятные названия
- забытые изменения

Лучше:

- делать маленькие коммиты
- использовать feature-ветки